Retriever: A Distributed Intrusion Detection System for NOS-Enabled Networks

Runmin Ou, Yijie Bai, Yanjiao Chen, *Senior Member, IEEE*, Bingchuan Tian, Ming Tang, Zhiming Ji, Ennan Zhai, Dennis Cai, and Wenyuan Xu, *Fellow, IEEE*

***Abstract*—Network Operating Systems (NOS) are being widely deployed on edge devices by cloud service providers to perform fast configurations and offer high availability for new network protocols. However, NOS-enabled networks open the door to intruders that can stealthily corrupt less-guarded programmable switches to launch attacks on the entire network. Traditional centralized intrusion detection systems may neglect anomalous events on NOS-equipped switches and fail to detect such attacks. In this paper, we make the first attempt towards intrusion detection for NOS-enabled networks by designing *Retriever*. *Retriever* features a lightweight local anomaly detection module on programmable switches and a central anomaly assessment module on the central server. The local anomaly detection module selectively traces both system and network events on switches, based on which a provenance graph of events is established. Upcoming events unmatched by the provenance graph are aggregated to construct a suspicious subgraph to report to the central server. The central anomaly assessment module extracts semantic representations from reported suspicious subgraphs and computes their anomaly scores. Large-scale experiments show that *Retriever* can achieve high intrusion detection accuracy (nearly 100%) with low overheads.**

***Index Terms*—Provenance graph, advanced persistent threat, intrusion detection, causality analysis, anomaly detection.**

I. Introduction

In RECENT years, Network Operating Systems (NOS) have been widely deployed on network switches by cloud service providers [6], [13], [54] for more flexible network functions [40], higher manageability [7], and fewer infrastructure costs [33]. Although NOS greatly enhances edge intelligence, it may also expose new attack surfaces. For example, dockers are widely used in open-source NOS for both typical switch control planes (e.g., routing protocols) and customized network functions (e.g., encapsulation). Unfortunately, such container applications may expose unsafe TCP/UDP ports to attackers [2] to gain control of the switches and perform further network intrusions [32], [47].

Intrusion Detection Systems (IDS) detect and identify potential security breaches in networks, generating real-time alerts and notifying administrators of the potential threat [35]. However, traditional intrusion detection systems are typically designed for servers but not for switches (or NOS). Burdensome data collection and computing-intensive analysis of existing IDS are not affordable for resource-limited switches. Other non-IDS protection methods, including firewall [45], malware detection [4], and kernel hardening [1], have difficulties in detecting more sophisticated adversary tactics and techniques like Advanced Persistent Threat (APT) [8].

To fill this research gap, in this paper, we present the design of *Retriever*, the first distributed intrusion detection system for NOS-enabled networks. *Retriever* features a local anomaly detection module deployed on switches and a central anomaly assessment module deployed on the central server, working cooperatively to perform intrusion detection.

Local anomaly detection: The NOS on switches has intensive CPU workloads for numerous tasks, including routing table calculation, network monitoring, network management, and even some complex data-plane functions. Therefore, the local anomaly detection module should not consume sizeable hardware resources (e.g., CPU, RAM, and even the disk). Otherwise, the critical functionality of switches could be severely affected. To address this challenge, we design and implement a lightweight local anomaly detection module to collect relevant events and build event provenance graphs on switches without influencing the normal NOS tasks (Section IV). The suspicious subgraphs will be extracted from provenance graphs and reported to the central server upon anomaly detection.

Central anomaly assessment: With a huge number of reported suspicious subgraphs from switches, the central server may be overwhelmed and fail to distill key information for intrusion detection. To address this challenge, we first extract informative semantic representations from suspicious subgraphs based on knowledge graphs and then compute both edge-level and graph-level anomaly scores to assess the anomaly degree of reported suspicious subgraphs. The feedback from network administrators on alerted suspicious subgraphs with high anomaly scores is utilized to continually improve *Retriever* with life-long learning.

We implement *Retriever* on our large-scale production cloud network, and the experiment results prove that *Retriever* can achieve accurate intrusion detection while keeping the CPU utilization ratio under 0.5% for event tracing, memory usage under

Received 30 September 2024; revised 28 September 2025; accepted 6 November 2025. Date of publication 21 November 2025; date of current version 12 March 2026. This work was supported in part by the project on Software Quality and Runtime Environment Security of Cloud Platform Domain Name Resolution, in part by the Zhejiang Key Laboratory of Electrical Technology and System on Renewable Energy, and in part by Alibaba Group through Alibaba Innovative Research Program. *(Corresponding authors: Yanjiao Chen; Ennan Zhai.)*

Runmin Ou, Yijie Bai, Yanjiao Chen, and Wenyuan Xu are with the College of Electrical Engineering, Zhejiang University, Hangzhou 310027, China (e-mail: ourunmin@zju.edu.cn; baiyj@zju.edu.cn; chenyanjiao@zju.edu.cn; xuwenyuan@zju.edu.cn).

Bingchuan Tian, Ming Tang, Zhiming Ji, Ennan Zhai, and Dennis Cai are with Alibaba Group, Hangzhou 311121, China (e-mail: bingchuan.tbc@alibaba-inc.com; ming.tang@alibaba-inc.com; zhiming.jzm@alibaba-inc.com; ennan.zhai@alibaba-inc.com).

Digital Object Identifier 10.1109/TDSC.2025.3635127

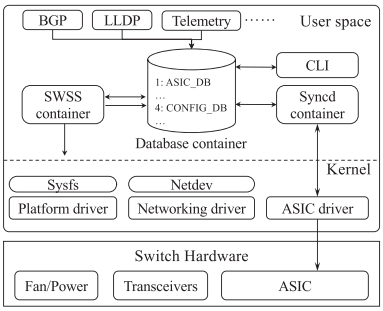


Fig. 1. SONiC, a Linux-based open-source NOS. It uses containers that manage different routing and control plane functions, which are delivered to ASIC to manage the data plane.

150 MB for the in-memory provenance graph construction, and network usage under 1 KB/s in each NOS.

We summarize our contributions as follows:

1) We make the first attempt to design a distributed intrusion detection system, *Retriever*, for NOS-enabled networks. *Retriever* can be deployed on programmable switches with limited resources.
2) We design a lightweight local anomaly detection module on switches to trace events and report suspicious subgraphs of provenance graphs. We develop a central anomaly assessment module on the central server to extract representations from provenance graphs and perform continuous intrusion detection with life-long learning.
3) We deploy *Retriever* on more than 50 switches in production environments, processing over 2 billion system events. We also validate *Retriever* on the DARPA dataset, and evaluation results show the superiority of *Retriever* over baselines.

II. Background & Motivation

In this section, we introduce the customized network operating systems and discuss potential attacks exploiting network operating systems, highlighting the need for a new paradigm of intrusion detection that taps into edge intelligence.

A. Network Operating Systems

There is a trend of edge intelligence in cloud computing. Traditionally, network switches are maintained by manufacturers and regarded as black-box routing devices by cloud service providers. To cater to the ever-increasing need for high-quality cloud services, cloud providers are transforming switches into white-box programmable devices using customized or open-source Network Operating Systems (NOS), e.g., SONiC [53]. SONiC is a Linux-based open-source NOS initially released in 2016 by Microsoft, actively developed and maintained by communities. As shown in Fig. 1, SONiC uses a containerized architecture for customized network protocol processing while Switch Abstraction Interface (SAI) sets up and manages the Application Specific Integrated Circuits (ASICs) for fast development. It uses containers that manage different routing and control plane functions. Such information is delivered to ASIC to manage the switch's data plane. The updates of routing tables and configurations are triggered by either a network operator (via Command Line Interface, CLI) or different network protocol processors, such as Border Gateway Protocol (BGP), Link Layer Discovery Protocol (LLDP), and telemetry. These updates are stored in the in-memory database (Redis) and then synchronized with the *syncd* container. Unlike traditional switches, users can access the CLI, log in to the backend Linux system, and even directly access the containers to manage customized network functions, which may expose programmable switches to higher security risks. Network operating systems enable cloud service providers to perform fast configurations and offer high availability for new network protocols and functions.

B. Security Concerns of NOS-Enabled Networks

Traditionally, network intrusion detection systems are deployed only at the central server but not edge switches [14], since the latter usually have limited programmable capabilities. With the rapid penetration of programmable switches equipped with NOS, new attack surfaces emerge as the attackers can stealthily corrupt the NOS at the edge to initiate attacks [30]. For example, high-privilege containers on NOS could be compromised by attackers leveraging vulnerabilities [46], allowing for container escaping. When compromised, the programmability of ASIC may further allow attackers to transform the switch into a high-performance network scanner [32], add malicious Access Control List (ACL) rules to block traffic, and announce fake Border Gateway Protocol (BGP) updates to paired switches, causing route hijacking. Specific examples are analyzed in Section VI-B. We give examples of advanced persistent threats (APT) that can potentially leverage NOS on programmable switches that differ from host-based scenarios in Section VI-B.

C. Intrusion Detection System

Intrusion detection systems monitor a network for malicious activities or policy violations. Intrusion detection and intrusion investigation are two typical functionalities of intrusion detection systems. Intrusion detection aims to detect ongoing intrusions, while intrusion investigation attempts to trace the provenance of detected intrusions [19].

Intrusion detection: Intrusion detection methods can be categorized as traffic-based and behavior-based. Distributed traffic-based IDS [16], [31], [39] has been widely explored, which monitors network traffic and extracts features for network attacks. Unlike existing distributed IDS in IoT, which are primarily traffic-based, our system is an event-based IDS that focuses on detecting and investigating the behaviors of the attacker. Behavior-based intrusion detection methods analyze potential malicious behaviors in host contexts (e.g., the system calls [15], process arguments and configurations [18]), using statistics-based, heuristic-based or learning-based approaches. DISTDET [11] extracts statistical characteristics from a system event tree to filter events and rank the anomaly of mismatched event sets. Heuristic algorithms have been designed based on specific system events [23], [41]. Recent works use deep learning to process raw logs or provenance graphs to identify anomalies [12], [21], [60].

Intrusion investigation: To cope with large-scale data logs, recent works on intrusion investigation mainly resort to techniques

that audit the occurrence of data provenance [19], [61]. Rule-based methods leverage the attacker's knowledge to endow specific meanings of suspicious actions. For example, researchers utilize anomaly score-based approaches for alert reduction [24] or rule-based attacker knowledge for alert correlation and analysis [23], [42]. Rule-based methods usually have poor generalization capabilities. Learning-based intrusion investigation method has a better generalization. Graph Neural Networks have been used to embed provenance graphs with sequence-based and recommendation-based architecture to analyze attacks [3], [60]. A major concern of provenance-based intrusion investigation is high time complexity due to a large amount of data provenance generated, which can lead to dependence explosion problems.

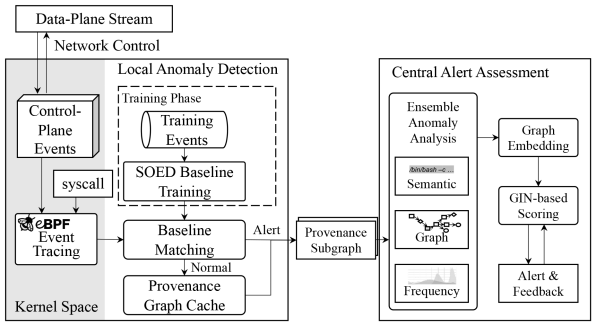


Fig. 2. Overview of *Retriever*.

III. System Overview

In this section, we define our threat model and outline the architecture of the proposed intrusion detection system.

A. Threat Model

We assume the APT scenarios for network system intrusion detection, extending existing host intrusion detection scenarios [22], [24], [28], [58]. An attacker illegitimately gains access to a NOS infrastructure, whose goals are persistently remaining there, exfiltrating sensitive information (e.g., network topology and routing tables), and impacting (e.g., for further network and host intrusion). The attacker may have significant knowledge and corresponding capability of controlling target NOS, where she/he is familiar with the common deployment patterns and management methods of the open-source NOS (e.g., container orchestration, SSH-based administration) and has the ability for conducting common attack techniques [8] as well as configure methods for control plane (e.g., CLI for modifying ACL rules and BGP routing configurations).

The goal of *Retriever* is to detect such attacks at any stage by analyzing the provenance generated by the local network devices. We assume the eBPF, which is the subsystem of the Linux kernel, correctly provides the monitor guarantees [51] for event collections. Like existing works [11], [22], we also make similar integrity assumptions for data collection and processing frameworks of *Retriever*.

B. System Overview

The goal of our proposed distributed intrusion detection system is to make a binary decision by comparing a constructed provenance graph G to a pre-encoded set of benign graphs [19],

$$f(G) = \mathbb{1}\left(\mathcal{F}\left(\mathcal{E}\left(\mathcal{N}(G)\right)\right) \geq \alpha\right), \tag{1}$$

where $G = (V, E)$ is a directed acyclic graph with the vertex set $V = \{v_i\}_{i=1}^{|V|}$ (representing process, file and network node) and the event set $E = \{e_j\}_{j=1}^{|E|}$ (corresponding to multi-level behaviors, like system calls and kernel or user activities). $\mathcal{N}$ is a deconstruction function that generates the suspicious subgraph for further analysis. $\mathcal{E}$ is the encoding function that encode the information in G. The encoding function greatly reduces irrelevant information in the raw graph G, which facilitates scalability for large-scale cloud networks. Finally, the central analysis function $\mathcal{F}$ performs an online analysis and generates the final alerts based on the scoring threshold α to network administrators.

To achieve this goal, we design *Retriever*, a distributed intrusion detection system, as illustrated in Fig. 2. The local anomaly detection module, deployed on the NOS system, collects and filters system behavior (including system calls and control-plane events), builds provenance graphs, and reports suspicious subgraphs to the central server. To determine whether the reported event is suspicious, we adopt a pretrained Subject-Object-Event Diagram (SOED) as a baseline model to check its behaviors and attributes. The central analyzer of the *Retriever* collects the reports from all local anomaly detection devices and constructs an ensemble anomaly analysis. The aggregated suspicious graphs are then fed into the ensemble anomaly detection model for anomaly detection and ranking. The anomaly alerts are reported to network administrators, including the graph with emphasized malicious nodes and their translated human-readable descriptions.

IV. Local Anomaly Detection

The local anomaly detection module features three carefully-designed sub-modules to suit resource-limited programmable switches.

A. Lightweight Event Tracing

Compared to traditional networks, NOS-enabled networks are susceptible to new attacks that leverage the large numbers of less-guarded programmable switches. Existing event tracing tools used for host-based intrusion detection systems (e.g., CamFlow [48] and Linux Auditd [50]) mainly focus on user behavior (i.e., interactions between kernel and users), which is not sufficient in handling security-related events [17]. Moreover, these techniques cannot capture underlay network events that are handled in kernel network stacks. For example, users are not aware of the ARP and BGP updates after changing the routing configuration. Existing event tracing tools include whole-system provenance tools (e.g., CamFlow [48]) and userspace tools (e.g., `tcpdump`), which log detailed information about data flows. While being suitable for comprehensive event tracing on central servers, these tools are extremely expensive for auditing on resource-limited edge devices [20], [49]. Therefore, we develop a lightweight event tracing tool for *Retriever*.

We trace three kinds of events (i.e., kernel functions, system calls, and netlink messages) which characterize system

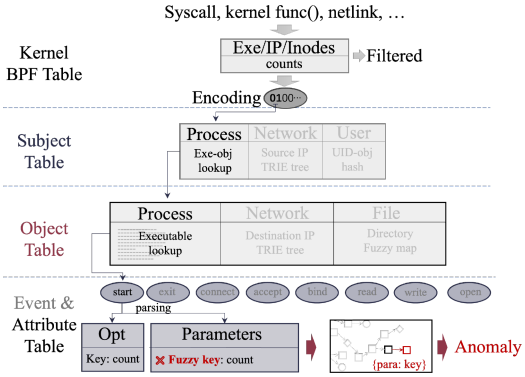


Fig. 3. Subject-Object-Event Diagram for Baseline Modeling of Provenance Graph.

behaviors including network events. As shown in Fig. 2, we design a lightweight event tracing engine based on the kernel network monitoring tool, i.e., the Extended Berkeley Packet Filter (eBPF). The eBPF runtime is a virtual machine system that supports static and dynamic probing for both kernel and user space. Probes are dynamic or static codes inserted at the beginning and end of kernel functions or system calls. They use a small set of helper functions carefully designed to minimize risk. Existing works have used eBPF for system behavior auditing with promising results [36], [51].

For monitoring NOS behaviors, we leverage reporting prioritization [51] and use another set of helper functions, which provide a key-value store called BPF tables, to filter the data to be reported. As illustrated at the top of Fig. 3, BPF tables define data structures that can perform exact string matches (using hash tables) or IP range matches (using long prefix-match trie trees). This strategy reduces the amount of log data, allowing Retriever to collect more critical events during busy states (as we evaluate in Section VII). In this way, we can flexibly define the BPF tables to limit the performance impact of the collection tool.

B. Baseline Modeling for Anomaly Detection

We build a space-efficient SOED baseline for anomaly detection, which only occupies a minimum space but preserves necessary details for suspicious events when an anomaly is detected. Our constructed baseline model is a Subject-Object-Event diagram (SOED), as illustrated in Fig. 3. The SOED consists of a hierarchical tree T and a series of lookup tables L. T contains four layers of nodes: Subject, Object, Event, and Attribute layers. The *subject* represents the process, either in userspace or in the kernel, which may have attributes including the process ID (PID), process command, owner, cgroup ID, and tags for data processing. The *object* includes the type (such as a file, network, tuple, or routing table), name, owner, and tags. The *edge* represents the events that may occur between a subject v_i and a vertex v_j (a subject or an object), i.e., $e_i = \langle v_i, act, v_j \rangle$. The *event* contains kernel hook functions and system calls (including clone and execute process, open and delete file, send and receive network, and so on), encoded with the arrangement in one byte to be routed to specific diagrams in matching. Each node in a layer links to a local or a global diagram L_k. We use on-demand tokenization for parameter generalization. For example, an executing event (`execve` syscall in UNIX) contains the parent process exe as the subject and the path of the executable file as an object to look up the exe hash table.

To save space, we perform event encoding as follows. Subject is encoded in 40 bytes (containing a 4-byte vertex identifier, 3-byte process ID, 16-byte for hash of executable path, 4-byte user id, 4-byte `cgroup` ID for different namespaces, an 8-byte timestamp for last event, and 1-byte tag for processing), while objects are encoded in 30 bytes (including vertex identifier, 1-byte object type, 20-byte object name, 4-byte owner id, and 1-byte tag). For event encoding, we limit to 7 bytes (with 1-byte type, 2-byte timestamp, and 4-byte object id). For network connection, the object is obtained from the subnet based on the IP trie tree, which fits NOS while a host-based system has broader IP ranges.

We initialize the baseline model using collected events from an attack-free network (as we assume in section Section III-A). During intrusion detection, we compare upcoming events with nodes on the baseline model. If there is a match, the event is considered benign and inserted into the provenance graph (as described in section Section IV-C); otherwise, the event is regarded as suspicious and used to build suspicious subgraphs. Given the current incoming traced event e'_i, we compare the subject, object, and event attributes of e_i with nodes on T. If there is a mismatch, the event will be tagged as suspicious as it is unseen during the training period.

C. Provenance Subgraph Reporting

Suspicious subgraph construction and reporting are triggered periodically or if the anomaly queue is full. The suspicious subgraph is composed of suspicious events and necessary benign events in the baseline model to associate these suspicious events. Similar to existing works [25], [28], we use forward tracking and backward tracking algorithms for suspicious subgraph generation, as described in Algorithm 1, with a highlight that the provenance graph is locally built. We report a space-efficient subgraph with detailed semantic information of the mismatched event for causal analysis. We first merge mismatched attributes att into a suspicious event of the subgraph G' (line 4), then perform forward tracking (lines 8-20) and backward tracking with half of the batch size. In forward tracking, we aim to obtain the entities after the mismatched event. We get the events whose timestamps are next to the current event (the first one in the candidate set C) and insert them into the subgraph (lines 9-11). Then, we search if there is the next event of the current subject and object (lines 15-18), and add them into C with the insertion sort algorithm. Furthermore, *Retriever* automatically merges consecutive events of the same type into a single edge and makes efforts to avoid object versioning. One example are illustrated in Fig. 4.

We use the decay rate τ and decay interval Δt to calculate the frequency of the subject-object-event diagram. For example, we set $\tau = 0.997$ and $\Delta t = 10$ s so that an entry in the anomaly queue will be forgotten in two days. The forgetting curve ensures that the same attack semantics within a short term are aggregated, so that the next attack can be triggered.

Algorithm 1: Suspicious Subgraph Generation.

Input: G: dependency graph; Q: anomaly queue; d: search depth; b: batch size;
Output: G': Suspicious Subgraph;

$G' = (V', E')$;
for each $entry$ **in** Q **do**
  $t_s, v, e, att \leftarrow entry$;
  $E' = E' \cup \{\langle e, att\rangle\}$
  $b_0 = 0$;
  /* Forward tracking */
  $C = \{\langle t_s, v, e\rangle\}$;
  repeat
    $t'_s, v', e' = C$.pop();
    $v'' = e'$.getObject();
    $V' = V' \cup \{v', v''\}$;
    if $e' \notin E'$ **then**
      $E' = E' \cup \{\langle e', \emptyset\rangle\}$;
    end if
    insertSort(C, G.getNextEvent(v', t'_s));
    if isSubject(v'') **and** getDistance(v, v'') $< d$ **then**
      insertSort(C, G.getNextEvent(v'', t'_s));
    end if
    b_0++;
  until $b_0 > b/2$ **or** isEmpty(C);
  $C = \{G$.getPreviousEvent(v, t_s) $\}$;
  /* Backward tracking */
end for
return G';

To prevent the provenance graph from growing infinitely large, we implement a pruning strategy. The pruning algorithm aims to delete benign nodes and edges that will not be needed in suspicious subgraph construction [22], such as those with no relationship to a suspicious event, to save memory space. The pruned provenance nodes will be stored on the disk for log persistence. Since the pruning algorithm needs to traverse through the provenance graph, we perform it periodically (e.g., every day) as a routine task with limited CPU usage.

V. Central Anomaly Assessment

The central alert assessment module conducts three steps on the central server to pinpoint network intrusions. First, we extract the semantic information of entities and events in suspicious subgraphs (reported by numerous switches) based on knowledge graphs. Second, we compute the anomaly scores of each suspicious subgraph based on the extracted representations. Third, we provide an in-depth analysis of suspicious subgraphs with anomaly indicators above the threshold.

A. Semantic Representation Extraction

The reported suspicious subgraphs contain implicit information that is useful for intrusion detection. To distill the key information, we propose to extract semantic representations of entities and events based on knowledge graphs [29], [56]. For a reported suspicious subgraph, we extract each edge to form the $e_i = \langle h, r, t\rangle$ tuple, where h is a subject entity, t is a subject or object entity, and r is the relation. We leverage the knowledge graph embedding methods TransE [5] to extract the embedding as semantic representations, maintaining the causal information in the tuples. We extract the semantic embedding representations separately for events and entities. In order to optimize the embedding representations, we set up scoring functions as the optimization target and maximize the score of the tuples in a training dataset. We utilize negative sampling to construct abnormal triples.

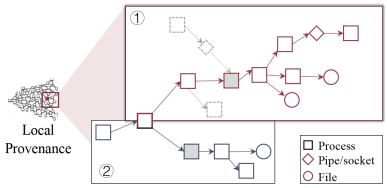


Fig. 4. Sub-graph construction for *Retriever* reporting.

B. Anomaly Score Calculation

After generating semantic representations for entities and events, we compute the anomaly score of each reported suspicious subgraph. We compute both the edge-based anomaly score and the graph-level anomaly score.

Edge-level anomaly score: The edge-level anomaly score assesses each edge in suspicious subgraphs. For each edge (h, r, t), we calculate the representation tuple (R_h, R_r, R_t). Then, we leverage the knowledge graph model M_K to predict the tail representation R_t based on the head representation R_h and relation representation R_r. The model will output each entity's possibility value P. The value of p_t for R_t demonstrates the possibility of R_t appearing given R_h and R_r. We use $1/p_t$ to indicate the edge-level anomaly score. We aggregate anomaly scores for all edges to form the global edge-level anomaly score $\mathcal{A}_e$ for the suspicious subgraph.

Graph-level anomaly score: With knowledge graph representation extraction model M_K, we transform the reported suspicious subgraphs $\{V, E\}$ into the representation subgraphs $\{V, E, R\}$. We leverage the representation graph analysis model Graph Isomorphism Networks (GIN) [57] to compute the graph-level anomaly score. Specifically, the GIN model defines a learnable aggregation function that combines the node's own features with the features of its neighboring nodes to generate new node representations. The design of this aggregation function enables the GIN model M_G to capture graph isomorphism, meaning that it can produce the same node representations for isomorphic graphs. The GIN model is updated as

$$h_v^{(k+1)} = \mathrm{MLP}^{(k)}\left(\left(1+\epsilon^{(k)}\right)\cdot h_v^{(k)} + \sum_{u\in\mathcal{N}(v)} h_u^{(k)}\right) \quad (2)$$

where $h_v^{(k)}$ represents the representation vector of node v at the k-th layer, $\mathcal{N}(v)$ denotes the set of neighboring nodes of v, $\mathrm{MLP}^{(k)}$ represents a multi-layer perceptron (MLP) [27] at the k-th layer, and $\epsilon^{(k)}$ is a learnable parameter. To train the GIN model, we construct a training dataset consisting of both benign and malicious subgraphs. The normal subgraphs are sampled from the provenance graph built in Section IV-B, labeled as 0. The malicious subgraphs are collected from the red team

attack rehearsal, labeled as 1. After training, the model M_G can predict the anomaly score $\mathcal{A}_g$ of a suspicious subgraph. When in an unsupervised setting where no attack subgraphs are provided, the graph-level anomaly assessment can be conducted with clustering.

C. Alert Analysis and Feedback

Suspicious subgraphs with anomaly scores higher than thresholds will be tagged as candidates for potential network intrusions and reported to human analysts for further analysis and processing. In large-scale clouds, analysts are usually fatigued by the huge number of alerts. To address this issue, we propose three strategies for better and more efficient alert analysis.

Alert aggregation: We leverage the graph embedding from GIN in Section V-B to extract graph embedding $\mathbf{e}_g$ from the alerted suspicious subgraphs as

$$\mathbf{e}_g^{(k)} = \mathcal{P}\left(h_1^{(k)}, h_2^{(k)}, \ldots, h_n^{(k)}\right), \tag{3}$$

where n is the number of nodes in the suspicious subgraph, $\mathcal{P}$ is the pooling function used in the GIN model, and k denotes the layer of the GIN model. After extracting the embedding for each alerted suspicious subgraph, we perform alert aggregation by embedding clustering. We divide the embeddings into several clusters with l_2 distance constraint less than d. For each cluster, we report the representative one to the analyst. The analyst may pre-define attack techniques by referring to ATT&CK. We can then match the alerted suspicious subgraphs with pre-defined attack techniques by the embedding distance.

Feedback to update Retriever: The analysis from human administrators will be the feedback to *Retriever* to improve its future performance. In particular, we update the knowledge graph model M_K and the GIN model M_G in the following way. To update the knowledge graph model M_k, we embed unseen entities from the feedback without influencing the former entity embedding. To achieve this goal, we implement the inductive knowledge graph embedding method [52], [55] to generate embedding for the emerging entities. After the completion, the neighborhood of the entity e_i is defined as $N(e_i)$, including all the entities having relations with e_i. To update the GIN model M_G, we aim to distinguish new training samples without forgetting prior training samples. Given the feedback samples, the GIN model M_G is fine-tuned with proper regularization on model parameters [34], [37] to avoid catastrophic forgetting.

VI. Deployment Experience

A. Implementation

We implement *Retriever* on 50+ testing and production network devices (white-box switches) with different network roles (including gateway and access switches) for anomaly detection testing. The reported sub-graphs of local machines are gathered via the gRPC tunnel and sent to the log service platform, which is like the Kafka service platform. The reported data are then fed into the central causal analysis system that runs on (settings) data processing cluster with typically an Intel Xeon CPU with 64 cores and 2.50 GHz and a memory of 256 GB.

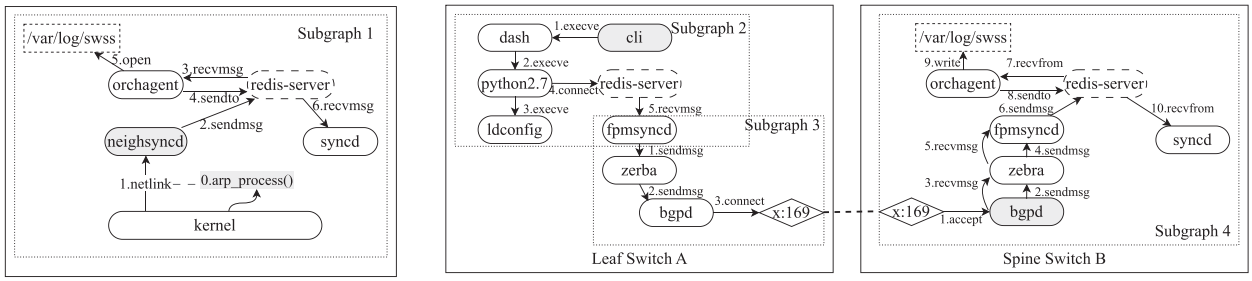
We use BPF Compiler Collection (BCC) for building system and network behavior collection. We developed 2K+ scalable BCC kernel code for kernel probe and tracepoint to hook kernel functions and syscall while using Redis API for network (e.g., routing tables) changes tracing. We built *Retriever* for local processing ($\sim 7.5K$ lines of Python code) and data processing cluster analysis ($\sim 5K$ lines of Python code and modules of a machine learning platform from our private cloud) for anomaly detection and causal analysis. The tracing tool is built in the container for the pluggable deployment, whose namespace is the same as the host for sidecar whole-system-wide tracing.

B. Case Study

To demonstrate the effectiveness of *Retriever*, we show two practical attacks that can possibly occur in the open-source NOS (SONiC as an example). We show that existing work tracing the system calls will need more information for network operators to understand such network attacks.

Case 1. spoofing attack: In the edge scenario, the NOS may face the public network, where adversary-in-the-middle (AitM) aims to spoof the NOS to obtain the opportunity for the following attacks [43]. Although neighbor spoofing attacks can occur in host devices, the kernel maintains such information and does not send any to userspace applications. Existing work cannot detect such attacks on the host since they only trace syscall. Fortunately, spoofing attacks cause different behaviors in network operating systems, which need to synchronize the neighbor state to the data plane. During the training phase, *Retriever*'s BCC collector traces the updated tables made by the kernel (using netlink message) and models its normal behaviors into SOED (e.g., encoded with ARP new or delete actions with attributes including MAC address and address-family). These neighbor-related netlink events triggered by newly discovered neighbors are sent by the kernel and processed by SNOiC for rewriting the adjacency table in the data plane (Fig. 1). During the detection phase, the kernel receives a malicious ARP message, and the unseen behavior of the ARP update will lead to an attribute mismatch (e.g., same IP address update with a different MAC address), which will trigger the *Retriever* to report a suspicious graph like Fig. 5(a). We show that our proposed method can protect against network attacks for protocols other than TCP and UDP in the network layer.

Case 2. traffic hijacking: Attackers who gained root access to the switch can modify the routing table for advanced network attacks [44]. Suppose that the attacker gains access to a white-box switch, and she/he wants to capture the packets that do not flow through the control plane (e.g., DNS packets that contain many fields and headers in which the attacker is interested). She/he generated a routing rule in CLI that the destination of the DNS server will forward to the interface of the control plane. The updated rule will trigger the BGP to announce the new route. The peer switches receiving such a message may perform a route execution and update their routing table to forward the DNS query to the compromised switch since it has a shorter length. Routing protocols have the characteristic that messages flow in one direction, and the traffic directed by them flows in the opposite direction. Without *Retriever*, network operators need much effort to find the root cause and may end up finding the misconfiguration of the routing table. Using the local reports and global analysis of *Retriever*, we can present the suspicious subgraph like Fig. 5(b), which connects the anomaly



(a) Subgraph of ARP spoofing.

(b) Subgraph of traffic hijacking.

Fig. 5. Case study of ARP spoofing and traffic hijacking.

TABLE I
OVERVIEW OF DATASETS

Dataset	Host Count	Day Count	Events (Mil)	Size (GB)	Average Size (B)	Attack Event
Private cloud	55	21	2,628	357.10	146	138
DARPA-Theia	3	11	51	5.76	113	95
DARPA-Cadets	3	11	7.85	80	98	47

of malicious rules and corresponding route updates via BGP connection. With this attack, a compromised switch can attract network traffic from the whole network. Also, such an attack can be achieved by not generating new routes but forwarding the BGP message to another Autonomous System (i.e., BGP leakage), which is more stealthy.

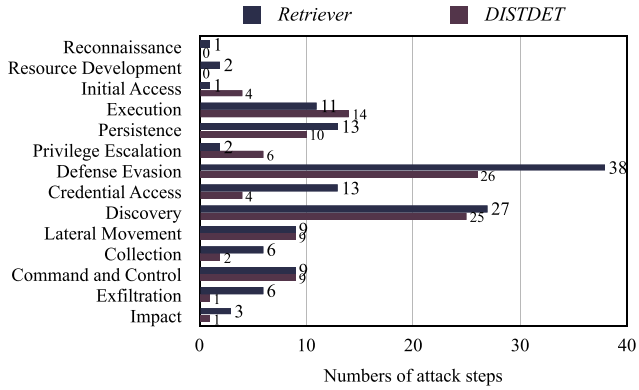


Fig. 6. Attack steps distribution.

VII. EVALUATION

Our experimental evaluation for *Retriever* is done in red team vs. blue team engagement. We also evaluate *Retriever* on public datasets for host intrusion detection and compare the detection accuracy with the state-of-the-art systems to evaluate the anomaly detection effectiveness. Note that the existing tagging and tracking datasets target detecting the interaction between the user and system (i.e., syscalls) while *Retriever* targets to detect the network attacks. We customize the *Retriever* to accommodate the detection of Advanced Persistent Threats (APTs) in host-based anomaly scenarios.

A. Evaluation Setup

Private cloud dataset: We conduct a 3-week-long adversary engagement rehearsal to evaluate the efficiency and effectiveness of our system. Due to the performance and transmission overhead, we cannot collect the complete auditing logs for the production area since it will dramatically affect the critical performance for logging and transferring such logs. To compare baseline systems that consume logs for anomaly detection, we collect the logs of one device for two days for baseline system training and the segments of the logs during the whole attack process. In total, we collected nearly 2.6 billion (357 GB) system and network events (as shown in the first row of Table I). The attacks from the red teams include host intrusion attacks and specific attacks (e.g., routing spoofing and container attacks). Some attack steps can be mapped into the MITRE ATT&CK matrix [8], as illustrated in Fig. 6. The red team performed ten attacks containing 138 attack steps. Compared with existing compacts (e.g., DISTDET [11] and DARPA), we also include the first two tactics (i.e., reconnaissance and resource development), where some attack steps target network devices. These attack steps are hard for the host intrusion system to detect, while the attackers open the way for subsequent attacks through these techniques.

Public dataset: The DARPA Transparent Computing (TC) Dataset is released by DARPA, aiming at earlier detection and causal analysis of activities across an enterprise [9]. Engagement#5 is done in a complex scenario for evaluating the design of tracking and analysis provided by different performers in May 2019 [10]. The DARPA TC Engagement 5 Dataset contains 12.7 billion events ($\sim$ 2.70TB) for 18 hosts in 11 days. We use two datasets (THEIA and CADETS) as the host dataset to compare the attack performance with state-of-the-art systems. THEIA (Tagging and Tracking of Multi-Level Host Events for Transparent Computing and Information Assurance) traces multi-level system and application behaviors by instrumenting the Ubuntu Linux systems. CADETS (Causal, Adaptive, Distributed, and Efficient Tracing System) collects data using DTrace from the FreeBSD systems. Table I shows the overview of the dataset. The APT attacks are performed in two to three days during the engagement. We convert the datasets into the Common Data Format provided by DARPA TC tools for better understanding and processing by *Retriever*. We split the dataset, using data from the first two days for baseline model training and knowledge graph extraction. The remaining data from each host are then

processed for testing. Since the DARPA datasets also record frequent events (i.e., file read and write), we filter out these events as if there is no reporting in the kernel ($\sim$ 70% of the dataset).

Ground truth: We label the attacks through the behaviors recorded by the red teams. We develop the scripts for labeling the ground truth in raw event logs. The scripts take configuration files as input according to the records of the red team (e.g., the ground-truth report generated by DARPA during Engagement 5), which contains the timestamp, host ID, session ID, command line, *etc*. We also analyze the network events in the ground truths and label the corresponding configurations (IP address, port, and direction) in the log files. Event labels are matched in the reporting subgraph of *Retriever* for detection result judgment. Note that we mark local and remote attacks separately, where the remote attack is the network access (e.g., Lateral Movement like ssh from the local network).

Metrics: We evaluate *Retriever* in cost and attack detection performance. We mainly focus on CPU overhead and network transmission reduction to evaluate the cost of monitoring an open-source network system. As for performance measurement, we use precision, recall, accuracy, and F1-score to evaluate attack detection performance because of the unbalanced number of attacks and benign sub-graphs. We consider the true positive as the sub-graph of final detection in the central analysis model containing a ground truth attack event and the false positive as the sub-graph with no attack events misclassified as an anomaly. Precision measures the proportion of true positive detections out of all positive detections made by the anomaly detection system. Recall is the proportion of true positive detections out of all actual attack instances. Accuracy measures the proportion of both true positives and true negatives out of all instances. F1-score is the harmonic mean of precision and recall, providing a single metric that balances both concerns.

B. Overall Results

We systematically conduct the entire processing pipeline of *Retriever* to evaluate its effectiveness. *Retriever* performed similarly in host intrusion detection scenarios and outperformed state-of-the-art intrusion detection systems in attack detection accuracy in the industry network scenarios.

Performance of private cloud dataset: The testing environment is complex. Deploying the baseline systems for real-time detection on network devices may affect normal operations. For fair comparison, we reserve the subgraphs and raw logs of the victim devices from the network device during the attack for offline comparison. The two-day-long raw logs of one device are collected for SOED (Section IV) and GIN (Section V) training. The same source is also trained for clustering and deep neural network training of the baseline systems. Overall results are shown in Table II. DeepLog and DeepCase could detect anomalies from raw logs via sequence modeling the event identifier after parsing by Drain [26]. They could achieve a nearly 100% recall rate but low precision(i.e., high false alarm rate). It is not affordable in practice, making many manual efforts to decide real attacks. Unicorn also uses a provenance graph for anomaly detection. Unicorn can detect 66% of the attacks in industry settings. This indicates that in an insufficient data setting, traditional host intrusion detection systems suffer from low detection rates. Our proposed system uses match-based anomaly detection and graph neural networks and can detect attacks under an unsupervised setting (normal data is obtained from the training data). We note that all the attack events are reported from the network devices. The reported specious sub-graphs indicate benign events and actual events. The unsupervised GIN is based on the feature distance to output the assessment, which can detect all the attacks during the engagement.

TABLE II
ATTACK DETECTION RESULTS ON DATASETS

Dataset	System	Precision	Recall	Accuracy	F1-score
Industry (Filtered)	DeepLog	12.45%	99.78%	12.59%	22.14%
	DeepCase	12.48%	**100.00%**	12.65%	22.19%
	Unicorn	65.47%	61.01%	85.71%	71.28%
	Retriever	**84.62%**	**100.00%**	**97.87%**	**91.67%**
Theia	Deeplog	8.97%	27.88%	87.31%	13.57%
	DeepCase	83.33%	26.98%	54.94%	40.76%
	Unicorn	**100.00%**	**100.00%**	**100.00%**	**100.00%**
	Retriever	93.46%	**100.00%**	99.77%	96.62%
Cadets	Deeplog	5.48%	11.63%	92.78%	7.45%
	DeepCase	5.09%	13.85%	83.87%	7.44%
	Unicorn	98.00%	**100.00%**	99.00%	98.99%
	Retriever	**100.00%**	**100.00%**	**100.00%**	**100.00%**

Performance of DARPA datasets: We also evaluate our algorithm on the DARPA datasets and compare it with the previous state-of-the-art algorithm as baseline systems. Table II (row 5 $\sim$ 12) shows the result in detecting host-based attack events. DeepLog and DeepCase achieve poor attack detection performance, although they achieve relatively high detection accuracy compared with that in industry settings. DeepLog has low precision, i.e., 8.97% for Theia and 5.48% for Cadets. This means that they can detect few true positive events and miss many attacks. DeepCase extends the capabilities of DeepLog by incorporating case-based reasoning and more sophisticated techniques for handling anomalies. Thus, DeepCase achieves relatively high performance compared to DeepLog in precision and recall. However, the accuracy of DeepCase is relatively low in our experiment settings. One possible reason is the unbalanced data distribution, and DeepCase is more concerned about the performance of detecting attacks. Unicorn can achieve nearly optimal performance in host attack detection, and *Retriever* can achieve comparable accuracy results as Unicorn. The high recall means the true threat samples can all be reported without missing. Due to the number imbalance between the actual threat samples and the normal operations, the accuracy is a little affected by the false threat samples, but also above 99% in both datasets, which proves the algorithm's effectiveness. The precision of *Retriever* on the Theia dataset is 93%, which means that there would be a few false positives (But no more than 10 for the entire 9-day-long compact). DeepLog and DeepCase have low precision in attack detection, indicating that deep learning directly on raw logs induces large false alarms.

Through the above analysis, event type-based anomaly detection directly on raw logs is insufficient in detection accuracy, especially in industry settings.

C. *Cost of* Retriever

We evaluate the CPU and network overhead for the cost of *Retriever*.

TABLE III
OVERHEAD OF COLLECTING FOR OVER ONE HOUR

Systems	CPU(%)	Avg. CPU(%)	MEM (MB)
Baseline (Network OS)	3.50 - 22.0	6.12	4030
Unicorn (CamFlow [48])	4.00 - 36.00	8.00	523
DARPA (Theia)	1.00 $\sim$ 69.00	16.00	1223
DARPA (Cadets)	13.00 $\sim$ 41.00	18.00	2200
Retriever (Linux Audit)	0.10 $\sim$ 6.60	0.80	128
Retriever (eBPF)	0.02 $\sim$ 0.20	0.05	154

TABLE IV
LOCAL MACHINE PERFORMANCE AND ATTACK DETECTION RESULTS OF INDUSTRY DATASET WITH DIFFERENT KERNEL BPF WHITELIST

ratio	CPU	Precision	Recall	Accuracy	F1-score
0×	0.051 %	84.38%	93.10%	96.67%	88.53%
1×	0.02%	84.62%	100.00%	97.87%	91.67%
2×	0.019%	87.50%	77.78%	93.75%	82.35%

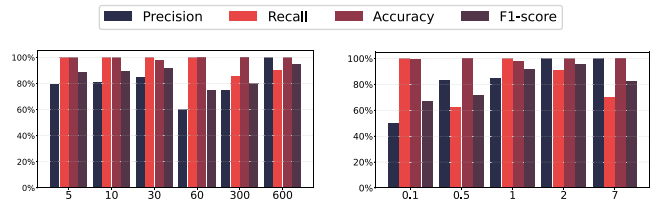
Computation cost on device: The network devices are performance-sensitive. Thus, the collecting and tracing function on the endpoint NOS needs to be lightweight. We evaluate the CPU overhead of the Linux Audit System and eBPF tracing tools of *Retriever*. As illustrated in Table III, when collecting network and process behaviors of NOS, the eBPF collector using the BTF table for filtering is more efficient than the rule matching of the Linux Audit System. Compared with the baseline (normal workload of NOS, CPU: 6.14%, Memory: 4.03 GB), the overhead of *Retriever* (about 150 MB) is small. Direct comparison of CPU cost is hard because we could not run all the systems on one device at the same time [22]. From the table, we can know that *Retriever* could achieve a largely low CPU utilization even with a higher average CPU on network devices. The eBPF is a subsystem within the Linux kernel, enabling direct and secure access to kernel structures and user data. By deploying our safelist within the kernel and utilizing BPF tables, we can reduce the communication between the kernel and userspace analysis tools. This reduction in communication consequently decreases CPU utilization.

Network communication cost: Retriever will report the suspicious subgraphs triggered by anomaly detection. On average, there are 45 thousand subgraphs generated in a 3-week-long engagement on the private cloud, with 4.05 KB for subgraph size on average. The average network overhead is within 0.24 KB/s, which is subtle compared with its large background traffic flow (e.g, more than 1 MB/s from data plane to control plane).

D. Influence of Report Analysis of Local

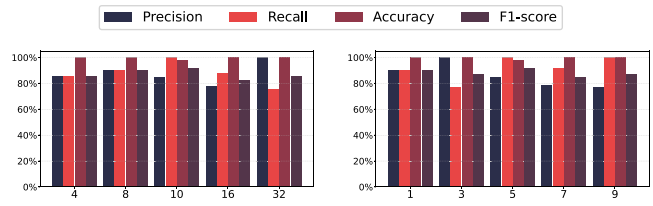
The local machine reports the suspicious sub-graphs to the central server for analysis. We now analyze the importance of the local machine's report interval and decay factors.

Kernel Filtering Effect: To reduce computation and network communication traffic, we utilize a kernel BPF table for lightweight event collection. We design the first-layer safelist in an industry setting to filter out specific events. Table IV presents the CPU overhead and corresponding anomaly detection results when no items are added and the number of items doubles. It is important to note that we need to define a few more regular expressions to handle the new events. Without the kernel BPF table, the CPU and network overhead increase by 150% and 800%, respectively.



(a) Report interval t (b) Decay day c for local caching

Fig. 7. Performance analysis for different parameters of *Retriever* that affect reporting timeliness.



(a) Branch number b of sub-graph (b) Search depth d for reporting

Fig. 8. Performance analysis for different parameters of *Retriever*.

Report interval t : Retriever will cache the anomaly events and trigger to analyze them for the sub-graph report. Fig. 7(a) shows the performance with the time interval between the reports. Smaller report intervals will cause more separated graphs and thus cause a large number of reports (up to 6 times). Compared with the existing system (e.g., 5500 s for Unicorn [22]), our proposed system can detect the anomaly with a much smaller time interval. Within 30 seconds or 1 minute, we can detect and report the alarms in a nearly real-time format. With longer report intervals, local machines can aggregate more normal and abnormal nodes. The reported sub-graph has more branches and depth, making the central analysis more accurate to detect the attack and benign activities, which means it has higher precision in the figure for a long report time of 10 minutes.

Decay parameters of local caching: We configured the decay parameters of caching based on the duration for which the SOED retains the new leaves. Fig. 7(a) presents the results when the decay days are set to 0.1, 0.5, 1, 2, and 7, respectively. With a short decay time, the local machine will report anomaly graphs more frequently (up to 5,000 per day per machine), leading to numerous false alarms, i.e., low precision. Conversely, a longer decay period will decrease recall, meaning that the local machine will not report similar anomalies.

E. Influence of Graph Analysis of Central

The alarm of anomaly attacks is highly related to the central Graph Isomorphism Networks model. We now analyze the importance of *Retriever*'s key parameters using industrial datasets in practice concerns.

Branch size b of sub-graph: Retriever aggregates the surrounding nodes to prepare the anomaly sub-graph for reporting. Fig. 8(a) shows the anomaly detection results with different branch sizes. The recall of detection initially increases but

TABLE V
PREDICTING RESULTS OF KNOWLEDGE GRAPH DATA

Dateset	KD	Precision	Recall	Accuracy
Theia & Cadets	0%	6.09%	37.05%	97.68%
	25%	20.01%	90.08%	87.75%
	50%	64.93%	92.59%	98.10%
	75%	90.51%	95.39%	99.52%
	100%	93.46%	100.00%	99.77%
Private cloud	0%	-	-	70.37%
	25%	72.73%	100.00%	88.89%
	50%	80.00%	87.50%	89.81%
	75%	96.97%	100.00%	99.07%
	100%	100.00%	100.00%	100.00%

TABLE VI
PREDICTING RESULTS WITH/WITHOUT LIFE-LONG LEARNING

	Precision	Recall	Accuracy
Partial	75.00%	56.25%	92.02%
With Life-long	84.62%	100.00%	97.87%

then decreases as the branch size grows. This indicates that normal nodes can help detect anomalies. However, too many normal nodes will confuse the detection model, where more weight is placed on surrounding nodes and edges, resulting in under-reporting. We choose a branch size of 10 for efficient reporting and stable detection.

Search depth d for reporting: For anomaly reporting, *Retriever* searches forward and backward from the unseen event to build the sub-graph. Fig. 8(b) illustrates the anomaly detection performance with different search depths. The larger depth of the graph enriches the graph connection of the surrounding nodes, which will help to detect more anomalies (with increased recall). When it exceeds 5, the larger depth of the report graph has no further effect on anomaly detection performance. It is reasonable because the most normal and attack process chain is within this depth; thus, looking backward through the suspicious node will usually have a maximum number of parent nodes.

Knowledge Effect: We conduct an ablation study to validate the effectiveness of the knowledge graph model. We apply changes to the knowledge graph model dataset by reducing it to 0%, 25%, 50%, and 75%. We show the results in Table V. As shown in the table, we can see that the knowledge graph's assistance mainly improves the threat detection precision by influencing the semantic-based anomaly assessment and graph-based anomaly assessment effectiveness. The absence of the private cloud data in 0% is because the model classifies no samples as the attack sample.

F. Life-Long Learning

In this part, we validate the life-long learning method results when *Retriever* faces changes in the production scenario. We show the results in Table VI. We construct a business change by introducing unseen entities and events into the private cloud dataset and evaluating the effectiveness of the normal algorithm. As seen in the table, the precision will drop rapidly because the unseen samples can easily be classified as threat samples. After forming the embedding for the unseen entities and events, the assessment models are retrained with the increased samples. The results show that after life-long learning, the model can increase the precision from 75.00% to 84.62%, and the recall from 56.25% to 100.00%, which means reducing the false threat samples and capturing the true threat samples.

VIII. DISCUSSION

Can Retriever defend resource exhaustion attacks? For resource exhaustion attacks like DDOS attacks, since they target exhausting the system resource rather brutally, they usually produce no malicious behavior. So, resource exhaustion attacks are out of our scope and can be prevented with conventional mitigation methods. *Retriever* can further combine the monitoring data on the CPU, memory, network, and FPGA usage for better defense against exhaustion attacks. It is exactly what the eBPF tools are born to do. In future research, we will explore integrating anomaly detection with machine performance observability to identify a broader range of network attacks.

Can Retriever defend adaptive evasion attacks? Attackers who have dissected the system's inner working mechanism may design adaptive attack methods to evade detection. For example, the attacker may replace the benign commands to pass the kernel caching. *Retriever* tries to detect anomalies in three submodels (process, file, network), and previously benign entities with unseen behaviors (like the connect system call) might be reported. Also, the adversarial attacks researched in recent years [38], [59] may also guide the attackers to work on bypassing the graph learning model. It exceeds the dependency graph's branch size and depth, resulting in attack step isolation, which may spoof the GNN model. The state-of-the-art AI model (i.e., Large Language Model, LLM) can help to analyze such advanced attacks. Large language models can be experts in judging malicious commands from normal, benign ones after fine-tuning, which is promising for network attack analysis.

IX. CONCLUSION

We present *Retriever*, the first practical, lightweight intrusion detection system for white-box switches in large-scale production environments. By the multi-layer design of local anomaly detection and centralized alert assessment, *Retriever* provides real-time switch monitoring and general comprehensive risk detection with high accuracy, saving the hard work of security and operation engineers.

In conclusion, *Retriever* stands out by integrating efficient local and centralized mechanisms, ensuring anomaly identification and ensemble feedback model response to potential threats.

REFERENCES

[1] M. Abubakar, A. Ahmad, P. Fonseca, and D. Xu, "SHARD: Fine-grained kernel specialization with context-aware hardening," in *Proc. Secur. Symp.*, 2021, pp. 2435–2452.
[2] A.-A. Agape, M. C. Danceanu, R. R. Hansen, and S. Schmid, "Charting the security landscape of programmable dataplanes," 2018, *arXiv: 1807.00128*.
[3] A. Alsaheel et al., "ATLAS: A sequence-based learning approach for attack investigation," in *Proc. USENIX Secur. Symp.*, 2021, pp. 3005–3022.
[4] Ö. A. Aslan and R. Samet, "A comprehensive review on malware detection approaches," *IEEE Access*, vol. 8, pp. 6249–6271, 2020.
[5] A. Bordes, N. Usunier, A. Garcia-Duran, J. Weston, and O. Yakhnenko, "Translating embeddings for modeling multi-relational data," in *Proc. Adv. Neural Inf. Process. Syst.*, 2013, pp. 2787–2795.

[6] Broadcom-Inc, "Enterprise SONiC," 2024. [Online]. Available: https://www.broadcom.com/products/ethernet-connectivity/software/enterprise-sonic

[7] Y. Chen et al., "Norma: Towards practical network load testing," in *Proc. USENIX Symp. Networked Syst. Des. Implementation* 2023, pp. 1733–1749.

[8] M. Corporation, "ATT & CK," 2024. [Online]. Available: https://attack.mitre.org/

[9] DARPA, "Transparent computing (Archived)," 2024. [Online]. Available: https://www.darpa.mil/program/transparent-computing

[10] DARPA, "Transparent computing engagement 5 data release," 2020. [Online]. Available: https://github.com/darpa-i2o/Transparent-Computing

[11] F. Dong et al., "DISTDET: A cost-effective distributed cyber threat detection system," in *Proc. USENIX Secur. Symp.*, 2023, pp. 1–18.

[12] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1285–1298.

[13] Edge-core, "SONiC–Open networking software for enterprise data center," 2024. [Online]. Available: https://www.edge-core.com/sonic.php,

[14] A. Febro, H. Xiao, J. Spring, and B. Christianson, "Edge security for SIP-enabled IoT devices with P4," *Comput. Netw.*, vol. 203, 2022, Art. no. 108698.

[15] S. Forrest, S. Hofmeyr, and A. Somayaji, "The evolution of system-call monitoring," in *Proc. Annu. Comput. Secur. Appl. Conf.*, 2008, pp. 418–430.

[16] A. R. Gad, M. Haggag, A. A. Nashat, and T. M. Barakat, "A distributed intrusion detection system using machine learning for IoT based on ton-IoT dataset," *Int. J. Adv. Comput. Sci. Appl.*, vol. 13, no. 6, pp. 548–563, 2022.

[17] A. Gehani and D. Tariq, "SPADE: Support for provenance auditing in distributed environments," in *ACM/IFIP/USENIX Distributed Systems Platforms and Open Distributed Processing*. Berlin, Germany: Springer, 2012, pp. 101–120.

[18] J. T. Giffin, D. Dagon, S. Jha, W. Lee, and B. P. Miller, "Environment-sensitive intrusion detection," in *Proc. Int. Symp. Recent Adv. Intrusion Detection*, 2006, pp. 185–206.

[19] A. Goyal, X. Han, G. Wang, and A. Bates, "Sometimes, you aren't what you do: Mimicry attacks against provenance graph host intrusion detection systems," in *Proc. Netw. Distrib. System Secur. Symp.*, 2023, pp. 1–18.

[20] B. Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd Edition. London, U.K.: Pearson Education, 2020.

[21] S. Han et al., "Log-based anomaly detection with robust feature extraction and online learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 2300–2311, 2021.

[22] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, "UNICORN: Runtime provenance-based detector for advanced persistent threats," in *Proc. Netw. Distrib. System Secur. Symp.*, 2020, pp. 1–19.

[23] W. U. Hassan, A. Bates, and D. Marino, "Tactical provenance analysis for endpoint detection and response systems," in *Proc. IEEE Symp. Secur. Privacy*, 2020, pp. 1172–1189.

[24] W. U. Hassan et al., "NoDoze: Combatting threat alert fatigue with automated provenance triage," in *Proc. Netw. Distrib. System Secur. Symp.*, 2019, pp. 1–15.

[25] W. U. Hassan, M. A. Noureddine, P. Datta, and A. Bates, "OmegaLog: High-fidelity attack investigation via transparent multi-layer log analysis," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–16.

[26] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. Web Serv.*, 2017, pp. 33–40.

[27] A. A. Heidari, H. Faris, S. Mirjalili, I. Aljarah, and M. Mafarja, "Ant lion optimizer: Theory, literature review, and application in multi-layer perceptron neural networks," *Nature-Inspired Optimizers: Theories, Literature Rev. Appl.*, vol. 811, pp. 23–46, 2020.

[28] M. N. Hossain et al., "SLEUTH: Real-time attack scenario reconstruction from COTS audit data," in *Proc. USENIX Secur. Symp.*, 2017, pp. 487–504.

[29] S. Ji, S. Pan, E. Cambria, P. Marttinen, and S. Y. Philip, "A survey on knowledge graphs: Representation, acquisition, and applications," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 2, pp. 494–514, Feb. 2022.

[30] P. R. Kannari, N. S. Chowdary, and R. L. Biradar, "An anomaly-based intrusion detection system using recursive feature elimination technique for improved attack detection," *Theor. Comput. Sci.*, vol. 931, pp. 56–64, 2022.

[31] R. Kumar, P. Kumar, R. Tripathi, G. P. Gupta, S. Garg, and M. M. Hassan, "A distributed intrusion detection system to detect DDoS attacks in blockchain-enabled IoT network," *J. Parallel Distrib. Comput.*, vol. 164, pp. 55–68, 2022.

[32] G. Li et al., "IMap: Fast and scalable in-network scanning with programmable switches," in *Proc. USENIX Symp. Networked Syst. Des. Implementation*, Renton, WA, USA, 2022, pp. 667–681.

[33] Z. Li et al., "A quantitative and comparative study of network-level efficiency for cloud storage services," *ACM Trans. Model. Perform. Eval. Comput. Syst.*, vol. 4, no. 1, pp. 1–32, 2019.

[34] Z. Li and D. Hoiem, "Learning without forgetting," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 12, pp. 2935–2947, Dec. 2018.

[35] H.-J. Liao, C.-H. R. Lin, Y.-C. Lin, and K.-Y. Tung, "Intrusion detection system: A comprehensive review," *J. Netw. Comput. Appl.*, vol. 36, no. 1, pp. 16–24, 2013.

[36] S. Y. Lim, B. Stelea, X. Han, and T. Pasquier, "Secure namespaced kernel audit for containers," in *Proc. ACM Symp. Cloud Comput.*, 2021, pp. 518–532.

[37] H. Liu, Y. Yang, and X. Wang, "Overcoming catastrophic forgetting in graph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 8653–8661.

[38] J. Ma, S. Ding, and Q. Mei, "Towards more practical adversarial attacks on graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 4756–4766.

[39] Q. Mao et al., "FeCoGraph: Label-aware federated graph contrastive learning for few-shot network intrusion detection," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 2266–2280, 2025.

[40] R. Miao, H. Zeng, C. Kim, J. Lee, and M. Yu, "SilkRoad: Making stateful Layer-4 load balancing fast and cheap using switching ASICs," in *Proc. Conf. ACM Special Int. Group Data Commun.*, 2017, pp. 15–28.

[41] S. M. Milajerdi, B. Eshete, R. Gjomemo, and V. Venkatakrishnan, "Poirot: Aligning attack behavior with kernel audit records for cyber threat hunting," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 1795–1812.

[42] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. Venkatakrishnan, "HOLMES: Real-time APT detection through correlation of suspicious information flows," in *Proc. IEEE Symp. Secur. Privacy*, 2019, pp. 1137–1152.

[43] MITRE, "Adversary-in-the-middle, technique T1157," 2020. [Online]. Available: https://attack.mitre.org/techniques/T1557,

[44] MITRE, "Automated Exfiltration: Traffic duplication, sub-technique T1020.001," 2020. [Online]. Available: https://attack.mitre.org/techniques/T1020/001

[45] K. Neupane, R. Haddad, and L. Chen, "Next generation firewall for network security: A survey," in *Proc. SoutheastCon*, 2018, pp. 1–6.

[46] N. I. of Standards and Technology, "CVE-2023–25809," 2023. [Online]. Available: https://nvd.nist.gov/vuln/detail/CVE-2023-25809

[47] S. Oliveira, F. Soares, G. Flach, M. Johann, and R. Reis, "Building a bitcoin miner on an FPGA," in *Proc. South Symp. Microelectronics*, 2012, pp. 1–4.

[48] T. Pasquier et al., "Practical whole-system provenance capture," in *Proc. Symp. Cloud Comput.*, 2017, pp. 405–418.

[49] T. Pasquier et al., "Runtime analysis of whole-system provenance," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 1601–1616.

[50] T. L. M.-P. Project, "auditd — linux audit daemon (manual page)," 2025, Accessed: Sep. 26, 2025. [Online]. Available: https://man7.org/linux/man-pages/man8/auditd.8.html

[51] R. Sekar, H. Kimm, and R. Aich, "eAudit: A fast, scalable and deployable audit data collection system," in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 87–87.

[52] B. Shi and T. Weninger, "Open-world knowledge graph completion," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 1957–1964.

[53] SONiC, "About SONiC," 2024. [Online]. Available: https://sonicfoundation.dev/about/

[54] Stordis, "SONiC-Powered networking for campuses: Making open networking and PoE as easy as pie," 2023. [Online]. Available: https://stordis.com/open-networking/sonic-powered-networking-for-campuses/

[55] P. Wang, J. Han, C. Li, and R. Pan, "Logic attention based neighborhood aggregation for inductive knowledge graph embedding," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 7152–7159.

[56] Q. Wang, Z. Mao, B. Wang, and L. Guo, "Knowledge graph embedding: A survey of approaches and applications," *IEEE Trans. Knowl. Data Eng.*, vol. 29, no. 12, pp. 2724–2743, Dec. 2017.

[57] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–17.

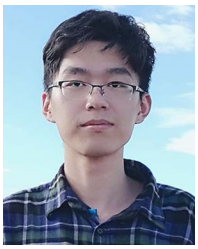
[58] C. Yagemann, M. A. Noureddine, W. U. Hassan, S. Chung, A. Bates, and W. Lee, "Validating the integrity of audit logs against execution repartitioning attacks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 3337–3351.

[59] X. Yin, W. Lin, K. Sun, C. Wei, and Y. Chen, "A2S2-GNN: Rigging GNN-Based social status by adversarial attacks in signed social networks," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 206–220, 2022.
[60] J. Zengy et al., "SHADEWATCHER: Recommendation-guided cyber threat analysis using system audit records," in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 489–506.
[61] M. Zipperle, F. Gottwalt, E. Chang, and T. Dillon, "Provenance-based intrusion detection systems: A survey," *ACM Comput. Surv.*, vol. 55, no. 7, pp. 1–36, 2022.

Runmin Ou received the BE degree in Internet of Things engineering and MS degree in software engineering from Wuhan University, China in 2020 and 2023, respectively. He is currently working towards the PhD degree with the College of Electrical Engineering, Zhejiang University, China. He majors in electronic information. His research interest includes network security, machine learning security, and authentication system.

Yijie Bai received the BE degree from the Department of Automation, Tsingshua University, China, in 2020 and the PhD degree from the College of Electrical Engineering, Zhejiang University, China. He is currently conducting research on large model security at Ant Group. His research interests include machine learning security, data privacy, and federated learning.

Yanjiao Chen (Senior Member, IEEE) received the PhD degree in computer science and engineering from the Hong Kong University of Science and Technology, in 2015. She is currently a bairen researcher with Zhejiang University, China. Her research interests include computer networks, wireless system security, cloud computing, and network economy.



Bingchuan Tian received the PhD degree from the Department of Computer Science and Technology, Nanjing University, China, in 2021. His research interests include network verification and network monitoring.

Ming Tang is a senior technical expert at Alibaba Infrastructure. His research focuses on the development of edge switches, hyper-converged gateway switches, and AI switches.

Zhiming Ji received the BE degree in information engineering from Shanghai Jiao Tong University, China in 2008. He currently works with Alibaba Cloud as a Network Operations Engineer in the network research and development team. His primary focus is on ensuring the stability and security of the underlying network infrastructure in large-scale cloud data centers through day-to-day operational activities.

Ennan Zhai is currently a Director of Network Research at Alibaba Cloud. Prior to joining Alibaba, he was a Research Scientist and a Lecturer with Yale University. He received the PhD degree from Yale University in 2015. His research focuses on building high-performance and reliable network systems for AI and Cloud, with a particular emphasis on network for AI and AI for network.

Dennis Cai photograph and biography not available at the time of publication.

Wenyuan Xu (Fellow, IEEE) received the BS degree in electrical engineering from Zhejiang University, in 1998, the MS degree in computer science and engineering from Zhejiang University, in 2001, and the Ph.D. degree in electrical and computer engineering from Rutgers University, in 2007. She is currently a professor with the College of Electrical Engineering, Zhejiang University. Her research interests include wireless networking, network security, and IoT security. She received the NSF Career Award in 2009, a CCS best paper award in 2017, and an ASIACCS best paper award in 2018. She was granted tenure (an associated professor) with the Department of Computer Science and Engineering, University of South Carolina, U.S. She has served on the technical program committees for several IEEE/ACM conferences on wireless networking and security, and she is an associated editor of TOSN.